

MP1 — Local Git Version Control Walkthrough

Prepared for: **Jaculbe** — folder `Jaculbe_local_project`, final file `MP1Jaculbe.pdf` Every command below was tested end-to-end on Git 2.43. Screenshot the terminal after each numbered step.

Step 0 — Verify Git and configure identity

```
git --version
git config --global user.name "Your Full Name"
git config --global user.email "youremail@example.com"
git config --global init.defaultBranch main
git config --global --list
```

 **Screenshot #1** — the output of `git --version`  **Screenshot #2** — the output of `git config --global --list`

Step 1 — Create the project folder and initialize the repo

```
mkdir Jaculbe_local_project
cd Jaculbe_local_project
git init
```

Expected output:

```
Initialized empty Git repository in .../Jaculbe_local_project/.git/
```

 **Screenshot #3** — the `git init` confirmation

Step 2 — Create initial files and make the first commit on `main`

Create `app.py`:

```
def greet():
    print("Hello from main branch")

if __name__ == "__main__":
    greet()
```

Create README.md :

```
# Jaculbe Local Project
```

Then:

```
git add .
git commit -m "Initial commit: add app.py and README"
git log --oneline
```

 **Screenshot #4** — git add, git commit, and git log --oneline

Step 3 — Create the `feature_x` branch

```
git checkout -b feature_x
git branch
```

Expected output:

```
Switched to a new branch 'feature_x'
* feature_x
main
```

 **Screenshot #5** — branch created and listed

Step 4 — Modify files, stage, and commit (commit #2)

Edit `app.py` so it reads:

```
def greet():
    print("Hello from feature_x branch")
```

```
def feature_x():  
    print("Experimental Feature X is running")  
  
if __name__ == "__main__":  
    greet()  
    feature_x()
```

Then:

```
git status  
git add app.py  
git commit -m "Add feature_x function and update greeting"
```

 **Screenshot #6** — `git status` before staging, then the commit result

Step 5 — A second commit on `feature_x` (commit #3)

```
echo "- Feature X: experimental module" >> README.md  
git add README.md  
git commit -m "Document Feature X in README"  
git log --oneline
```

 **Screenshot #7** — two commits now visible on `feature_x`

Step 6 — Switch back to `main`

```
git checkout main  
cat app.py
```

Note that `app.py` reverts to the original version — this demonstrates branch isolation.

 **Screenshot #8** — the switch plus the reverted file contents

Step 7 — Create a conflicting change on `main` (commit #4)

This step is what *guarantees* a merge conflict, so you can show conflict resolution.

Edit `app.py` on `main` to:

```
def greet():
    print("Hello from the stable main branch")

if __name__ == "__main__":
    greet()
```

Then:

```
git add app.py
git commit -m "Update greeting message on main"
git log --oneline
```

 **Screenshot #9** — the divergent commit on `main`

Step 8 — Merge `feature_x` into `main` (conflict appears)

```
git merge feature_x
```

Expected output:

```
Auto-merging app.py
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and then commit the result.
```

```
git status
cat app.py
```

The file will contain conflict markers:

```
def greet():
<<<<<<< HEAD
    print("Hello from the stable main branch")
=====
    print("Hello from feature_x branch")

def feature_x():
    print("Experimental Feature X is running")
>>>>>>> feature_x

if __name__ == "__main__":
```

```
greet()
feature_x()
```

🔥 **Screenshot #10** — the CONFLICT message 🔥 **Screenshot #11** — the file showing <<<<<<, =====, >>>>>> markers

Step 9 — Resolve the conflict

Delete the three marker lines and keep the combined result:

```
def greet():
    print("Hello from the stable main branch")

def feature_x():
    print("Experimental Feature X is running")

if __name__ == "__main__":
    greet()
    feature_x()
```

Then:

```
git add app.py
git commit -m "Merge feature_x into main and resolve conflict in app.py"
```

🔥 **Screenshot #12** — resolved file + merge commit

Step 10 — View the commit history

```
git log --oneline --graph --all
```

Expected output:

```
*   bdad0df Merge feature_x into main and resolve conflict in app.py
| \
| * e567f46 Document Feature X in README
| * 79a95a2 Add feature_x function and update greeting
* | 5e41e66 Update greeting message on main
| /
* 462c559 Initial commit: add app.py and README
```

Also useful for the writeup:

```
git log --stat
```

 **Screenshot #13** — the branching graph. This is the single most important screenshot in the submission.

Step 11 — Tag version v1.0

```
git tag -a v1.0 -m "Version 1.0 - first stable release"
git tag
git show v1.0 --stat
```

 **Screenshot #14** — tag created and shown

Step 12 — Run the program

```
python3 app.py
```

Expected output:

```
Hello from the stable main branch
Experimental Feature X is running
```

 **Screenshot #15** — working output of the merged code

Document Structure (for the Word file → PDF)

Save the final file as `MP1Jaculbe.pdf` — wrong filename costs 10 points.

Title page

- Machine Problem #1 — Local Git Version Control
- Name, Section, Subject, Date

I. Objective — paste the objective from the handout

II. Problem Statement — paste from the handout

III. Environment Setup — Screenshots 1–4

IV. Tasks

- 4.1 Creating a new branch → Screenshot 5
- 4.2 Staging changes → Screenshot 6
- 4.3 Committing changes → Screenshot 7
- 4.4 Switching back to main → Screenshot 8
- 4.5 Divergent commit on main → Screenshot 9
- 4.6 Merging the feature branch → Screenshots 10, 11
- 4.7 Conflict resolution → Screenshot 12
- 4.8 Viewing commit history → Screenshot 13
- 4.9 Tagging v1.0 → Screenshot 14

V. Source Code — full `app.py` and `README.md`, in a monospace font (Consolas 10pt) with a light grey background

VI. Output — Screenshot 15

VII. Conclusion — 3–5 sentences on what branching, merging, and conflict resolution accomplish

Formatting tips

- Body text: Arial or Times New Roman 11–12pt
- Code blocks: Consolas / Courier New 10pt, left-aligned, never justified
- Caption every screenshot: *Figure 1. Git version and global configuration*
- Crop screenshots to just the terminal window — no desktop background
- Save via **File → Save As → PDF**, not by printing to PDF, so text stays selectable